

The Engineering of Scale: A Technical Analysis of WhatsApp Architecture and Systems

The architectural trajectory of WhatsApp is a masterclass in distributed systems engineering, defined by an uncompromising commitment to simplicity, vertical scaling, and the selection of technologies based on mathematical efficiency rather than industry fashion. At its core, the platform's technical personality is shaped by a "Zero Trust" model concerning the server layer, where infrastructure is treated as a high-performance delivery mechanism that remains structurally incapable of accessing the content of user communications. This report provides an exhaustive technical examination of the protocols, frameworks, and engineering philosophies that enable a relatively small team of engineers to support over 3 billion active users processing over 100 trillion messages annually.

Architectural Foundations and Engineering Philosophies

The engineering ethos of WhatsApp is rooted in architectural restraint. Unlike contemporary organizations that often adopt microservices early in their lifecycle, WhatsApp famously scaled to hundreds of millions of users using a largely monolithic Erlang-based backend. This decision was driven by the need for operational simplicity; a smaller codebase managed by a high-context team of engineers reduces the coordination overhead typically associated with large-scale distributed systems.

Reliability is the primary functional requirement, predicated on the principle that messages must be delivered with sub-second latency regardless of network conditions. This necessitates a "mobile-first" networking strategy, engineered specifically for high-latency, low-bandwidth environments such as 2G and EDGE networks. The system handles this through a combination of lightweight persistent connections and ultra-efficient binary protocols.

The "Let It Crash" philosophy, inherited from Erlang/OTP, dictates the system's fault-tolerance model. Instead of writing exhaustive defensive code, the architecture isolates every user connection into its own process. If a bug occurs, only that specific process fails, and a supervisor process immediately restarts it in a clean state, ensuring that failures are localized and self-healing.

The Backend Core: Erlang/OTP and the BEAM Virtual Machine

The choice of Erlang as the primary programming language is the single most critical factor in WhatsApp's ability to handle massive concurrency. Erlang was designed by Ericsson for telecommunications switches, making it uniquely suited for the "C10M" problem—managing ten million concurrent connections on a single server.

Lightweight Concurrency and the Actor Model

The Erlang runtime, known as the BEAM Virtual Machine, utilizes lightweight processes rather than operating system threads. A traditional OS thread can consume up to 1 MB of memory, whereas an Erlang process starts with as little as 2 KB of memory. This allows a single commodity server with 64GB of RAM to host over 2 million concurrent connections, each monitored by its own isolated process.

Feature	Traditional Thread-Based Model	Erlang/BEAM Actor Model
Concurrency Primitive	OS Threads	Lightweight Green Processes
Memory Overhead	~1 MB per thread	~2 KB per process
Context Switching	Kernel-level (Expensive)	VM-level (Highly efficient)
Communication	Shared memory with locks	Asynchronous message passing
Fault Isolation	Single thread crash can kill process	Total process isolation

Each user session is an "Actor" that maintains its own state and communicates with other actors via message passing. This avoids the complexity of semaphores and mutexes, which frequently become bottlenecks in high-scale Java or C++ environments.

Hot Code Swapping and Operational Continuity

One of Erlang's most distinctive features utilized by WhatsApp is hot code swapping. The BEAM VM can load and execute new code while the application is still running. In the WhatsApp deployment, active processes continue running the old version of the code until they reach a quiescent state or a logic checkpoint, at which point they seamlessly transition to the new version. This allows the engineering team to deploy updates, patches, and new features without disconnecting millions of users or requiring service windows.

Operating System and Network Stack: The FreeBSD Choice

While Linux is the standard for most cloud-scale applications, WhatsApp famously chose FreeBSD for its backend infrastructure. This decision was based on the performance of the FreeBSD networking stack and its kqueue event notification system, which proved more stable and efficient for long-lived, largely idle socket connections compared to Linux's epoll at the time.

Kernel Tuning for Massive Scalability

The achievement of 2 million connections per host required deep kernel-level optimizations in FreeBSD. The engineers tuned parameters such as `kern.ipc.nmbclusters` to increase the number of network buffer clusters and `net.inet.tcp.maxtcptw` to manage TCP time-wait states. The team also optimized the TCP Initial Window (IW). While the IETF recommendation is 10 segments, increasing this value allowed WhatsApp to send more data in the first round-trip time, significantly improving performance for small message transactions. Furthermore, by utilizing FreeBSD's Zero Copy socket options and tuning the memory allocation for mbuf clusters (each

requiring 2KB of memory), the team maximized throughput while minimizing CPU cycles spent on memory management.

FunXMPP: The Messaging Protocol and Binary Trees

WhatsApp utilizes a proprietary messaging protocol named FunXMPP, which is a highly compressed version of the Extensible Messaging and Presence Protocol (XMPP). Standard XMPP transmits data as verbose XML text, which is suboptimal for mobile networks where 200 bytes of XML tags might be used to deliver only 5 bytes of actual content.

Tokenization and Compression Dictionary

FunXMPP replaces common XML strings with single-byte tokens. The protocol employs a static dictionary of over 200 predefined strings. For example, the string "message" is mapped to the byte 0x59, and common attributes like "type" and "from" are similarly tokenized.

XML Element/Attribute	Standard Representation	FunXMPP Token
message	<message>	0x59
presence	<presence>	0x16
id	id=	0x11
type	type=	0x14
@s.whatsapp.net	Domain String	0x91

This tokenization reduces a typical 180-byte XML message to approximately 20 bytes. The protocol also optimizes Jabber IDs (JIDs) using special handling for the user@server format, where a single JID_PAIR token (250) indicates the structure, further cutting bandwidth.

Binary Node Structure

Beyond tokenization, FunXMPP eliminates the structural overhead of angle brackets and closing tags. It uses a binary tree format where a single byte indicates the size of a list. For instance, the byte 0xF8 might signal that three items follow. The parser counts these items rather than searching for closing tags like </message>, reducing CPU-intensive string parsing. This protocol is transmitted over a persistent TCP socket, often using the Noise Protocol Framework for the transport layer security instead of standard TLS, which reduces handshake overhead.

The Security Architecture: Signal Protocol and Multi-Device Sync

The privacy model of WhatsApp is built on end-to-end encryption (E2EE) using the Signal Protocol. This ensures that messages are encrypted on the sender's device and can only be decrypted by the recipient's device.

Key Agreement via X3DH

WhatsApp uses the Extended Triple Diffie-Hellman (X3DH) protocol to establish a shared secret between users who may be offline. Upon installation, the app generates a "Pre-Key Bundle"

consisting of:

1. **Identity Key:** A long-term Curve25519 key pair.
2. **Signed Pre-Key:** A medium-term key pair signed by the Identity Key.
3. **One-Time Pre-Keys:** A batch of keys intended for single use, which the server deletes after they are used to establish a session.

The shared secret is derived using a series of Diffie-Hellman (DH) operations between these keys, ensuring that even if the server is compromised, it cannot derive the encryption keys.

The Double Ratchet Algorithm

Once a session is established, WhatsApp employs the Double Ratchet algorithm for ongoing encryption. This combines a Diffie-Hellman ratchet with a symmetric-key ratchet based on a Key Derivation Function (KDF).

- **Forward Secrecy:** Each message key is derived from a chain key using a KDF. Once a message is sent, the chain key is ratcheted forward, making it mathematically impossible to derive previous message keys from the current one.
- **Post-Compromise Security:** Every message includes a new ephemeral DH public key. When a recipient replies, a new DH exchange occurs, updating the "Root Key" of the KDF chain. If an attacker steals a device's keys, they can only decrypt messages until the next DH ratchet turn occurs.

Multi-Device Management: The Sesame Algorithm

The 2022 multi-device architecture allows up to four companion devices to operate independently of the primary phone. This transition was one of the largest engineering rewrites in the company's history. It relies on the Sesame algorithm to manage encryption sessions in an asynchronous environment.

Each device (phone, tablet, or desktop) has its own unique identity key pair. When Alice sends a message to Bob, her device must encrypt the message separately for each of Bob's registered devices. The Sesame algorithm ensures that all devices "converge" on the correct active session. It handles "stale" device records by marking them for deletion after a period of inactivity, ensuring the encryption state remains manageable.

Data Layer and Storage Infrastructure

WhatsApp's storage architecture is a hybrid system designed for "at-least-once" delivery and extreme write throughput.

Mnesia: Distributed Metadata Store

Mnesia is an Erlang-native, distributed DBMS used for low-latency lookups. It handles real-time session information, routing tables (mapping which server handles which user), and group memberships. Because Mnesia stores data in Erlang's native term format, there is zero serialization/deserialization overhead when the application interacts with the database.

MySQL and MyRocks (RocksDB)

For persistent storage, such as user profiles and the message queue for offline users, WhatsApp uses a sharded MySQL architecture. To improve storage density and write performance, Meta integrated RocksDB as a storage engine for MySQL, a project known as MyRocks.

Database Engine	Performance Characteristic	Advantage for WhatsApp
InnoDB (Standard)	B+ Tree based	High read performance, but high write amplification
MyRocks (RocksDB)	LSM Tree based	Space-optimized, handles massive random writes on HDD/SSD
Mnesia	RAM-first / Erlang native	Sub-millisecond session lookups with no ORM lag
SQLite	Embedded / Client-side	Persistent local chat history, reducing server load

MyRocks utilizes Log-Structured Merge-trees (LSM), which are significantly more efficient for the write-heavy workloads of a global messaging app. It offers approximately 2x better compression than InnoDB, which is critical when storing billions of metadata records.

Message Persistence and Delivery Guarantees

WhatsApp does not store messages permanently on its servers; they are deleted immediately upon successful delivery. For offline users, messages are stored as encrypted blobs in persistent queues. The system uses FIFO logic for delivery and relies on device-side deduplication, meaning the server sends every message in the queue and the client handles any duplicates caused by network retries.

The MEX Layer: Modern Microservices and Meta Integration

Following its acquisition by Meta, WhatsApp's architecture has evolved to include a "MEX" (Meta Experience) layer. MEX is a high-trust, internal GraphQL layer that handles the platform's most sensitive and complex operations, such as regulatory compliance and cross-service identity linking.

GraphQL Mesh Architecture

WhatsApp operates a distributed mesh of specialized GraphQL gateways. This allows for security isolation; for instance, generative AI mutations (like CreateMEMuProfile) are handled by dedicated backends that have no access to the core messaging databases or contact lists.

- **MEX Schema (whatsapp_mex):** Handles higher-trust operations like EU Digital Markets Act (DMA) interoperability, parent-child account linking, and passkey authentication.
- **Public Schema (whatsapp):** Manages general messaging features, payments (UPI/Pix), and AI interactions.
- **Persisted Queries:** To save bandwidth, clients do not send full GraphQL query strings. They send a hash of the query, and the server looks up the query text from a pre-registered dictionary, executing it with the provided variables.

DMA Interoperability Framework

To comply with European regulation, the MEX layer supports third-party messaging interoperability. It includes operations like `GroupsCreateInteropGroup` and `AddParticipantsToInteropGroup`, which allow WhatsApp users to create mixed-platform groups containing contacts from Signal or Telegram. This architecture implies a federation layer that translates between different cryptographic protocols while maintaining E2EE as much as possible.

Identity Proof Linked Storage (IPLS) and Contact Management

Contact management was historically a "mobile-only" feature, but the introduction of IPLS allowed for private contact storage across all linked devices.

Cryptographic Contact Vaulting

IPLS allows users to save contacts exclusively to WhatsApp in an encrypted format.

1. **Encryption:** Contact names are encrypted on the device using a symmetric key generated by the client.
2. **Hardware Security Modules (HSMs):** These encrypted names are stored in an HSM-based Key Vault in Meta's data centers. HSMs provide a tamper-resistant environment where even Meta administrators cannot see the keys.
3. **Auditable Key Directory (AKD):** WhatsApp uses a key transparency system to verify device identities. Any changes to a user's identity keys are published to an append-only cryptographic directory.
4. **Cloudflare Witness:** To prevent Meta from silently tampering with the AKD, Cloudflare acts as an independent third-party witness, digitally signing each directory update to prove it has not been altered.

Kaleidoscope and the Transition to Rust

A major recent evolution in the WhatsApp engineering stack is the migration of high-risk media handling code from C++ to Rust. This effort was a direct response to vulnerabilities like "Stagefright," which showed that malware could be hidden in benign media files like MP4s.

The Media Consistency Library

The **Kaleidoscope** system is a media consistency library written in Rust that validates media files before they are processed by potentially vulnerable OS libraries.

Metric	Legacy C++ Library (wamedia)	New Rust Library (Kaleidoscope)
Lines of Code	~160,000	~90,000
Safety Model	Manual memory management	Memory-safe by default
Primary Goal	Format/Send MP4s	Detect

Metric	Legacy C++ Library (wamedia)	New Rust Library (Kaleidoscope)
		malicious/non-conformant files
Testing Method	Unit/Integration	Differential Fuzzing

WhatsApp engineers used differential fuzzing to ensure that the Rust version behaved identically to the C++ version for all valid inputs, while correctly rejecting the malformed inputs that might trigger exploits. The Rust version showed performance and runtime memory usage advantages, disproving the notion that memory-safe languages necessarily introduce a "performance tax".

Build System and Deployment

Kaleidoscope represents one of the largest client-side deployments of Rust code in history, shipping to billions of devices across Android, iOS, Windows, macOS, and web. To manage the complexity of compiling Rust for dozens of target architectures while keeping binary sizes small, WhatsApp transitioned its build system to **Buck2**. Buck2 allowed for faster incremental builds and helped minimize the Rust standard library's overhead to approximately 200 KiB initially.

Scaling Infrastructure and Message Delivery Lifecycle

The "instant" feel of WhatsApp is a result of extreme optimization in the message delivery lifecycle.

Instant Local Acknowledgement

When a user taps "Send," the app immediately records the message in the local SQLite database and displays a single grey tick. This "local acknowledgement" creates the illusion of speed, as the UI doesn't wait for a network response to update.

Persistent Connections and MQTT

Unlike standard web applications that use HTTP polling, WhatsApp maintains a long-lived TCP connection. It often utilizes a lightweight version of the MQTT (Message Queuing Telemetry Transport) protocol to keep the connection open with minimal battery drain. This persistent socket eliminates the 100-300ms delay required for a new TLS handshake on every message.

Global Edge Routing

WhatsApp utilizes Meta's global network of edge servers. When a message leaves a device, it is routed to the nearest edge location, which then forwards the encrypted blob over Meta's high-speed private backbone to the appropriate chat server shard.

Message Fan-out for Groups

In group messaging, WhatsApp performs the "multiplication" on the server side. When a user sends a message to a group of 100 people, their phone uploads only one encrypted copy of the message. The server-side Erlang process for that user identifies all 100 recipients and handles

the delivery to their respective connection processes simultaneously across the cluster. This saves significant battery and data for the sender.

Client-Side Architecture and Frameworks

WhatsApp develops native applications for each platform (Android, iOS, Windows, macOS) rather than using hybrid frameworks, ensuring maximum performance and deep integration with OS-level push notification services like Firebase Cloud Messaging (FCM) and Apple Push Notification service (APNs).

Local Persistence: SQLite and IndexedDB

The primary data store on mobile devices is SQLite, a self-contained relational database embedded into the app. This allows chat history to be searchable and accessible even when the device is offline. For WhatsApp Web, the app uses IndexedDB, a browser-based storage API that functions as a digital filing cabinet for messages and media thumbnails, as local storage (LocalStorage) is too limited for the volume of data generated by modern chats.

Desktop and Web Frameworks

The newer version of the WhatsApp Desktop app has seen architectural shifts. While some components are built using Electron (a framework for building desktop apps with JavaScript and Chromium), native components remain critical for performance-heavy tasks like media rendering and VOIP calls. Research also indicates the existence of native protocol implementations in languages like Rust, which provide significantly lower memory footprints (binary size of ~20MB) compared to Electron-based alternatives.

VOIP and Multimedia Infrastructure

WhatsApp's voice and video call architecture is designed to bypass the traditional telecommunications grid, relying instead on a global network of STUN (Session Traversal Utilities for NAT) and TURN (Traversal Using Relays around NAT) servers.

Call Establishment and SRTP

1. **Signaling:** When a call is initiated, an encrypted FunXMPP signaling message is sent to the recipient, containing a random 32-byte SRTP (Secure Real-time Transport Protocol) master secret.
2. **Negotiation:** The devices use STUN/TURN servers to negotiate a direct peer-to-peer connection. If a direct connection is impossible due to restrictive firewalls, the call is relayed through a WhatsApp media server.
3. **Encryption:** Once the connection is established, the media stream is encrypted using the SRTP master secret, ensuring the call is end-to-end encrypted.

Multimedia Blob Storage

Media files (images, videos, documents) are handled differently than text messages.

- **Encryption:** The sender's device generates a random AES-256 key and encrypts the media file locally.
- **Blob Store:** The encrypted file is uploaded to a blob store (often Meta's "Everstore" or a global CDN like Amazon S3).
- **Pointer Delivery:** The sender then sends a small FunXMPP message containing the URL of the blob and the decryption key. The recipient downloads the blob and decrypts it locally. This separation allows WhatsApp's core Erlang servers to remain lightweight, as they never handle large binary data directly.

Technical Personalities: The Engineering Spirit of WhatsApp

The architectural success of WhatsApp is inseparable from its technical culture. The engineers exhibit a personality characterized by "Ruthless Pragmatism".

Ruthless Pragmatism and Small Teams

WhatsApp is famous for supporting 450 million users with only 32 engineers at the time of its acquisition, and even today, the core infrastructure team remains remarkably small given the service's scale. This was possible because the team resisted "Feature Bloat". Every new feature is scrutinized for its impact on the system's simplicity. For example, the team spent months benchmarking FreeBSD vs. Linux under realistic loads rather than choosing the more popular OS based on intuition.

Engineering for Resilience over Prevention

The team assumes that everything will fail: networks will drop, servers will crash, and datacenters will lose power. This engineering personality leads to a system that prioritizes recovery speed over failure prevention. The "Let it crash" model and the use of persistent queues for retries are direct results of this mindset. During the massive Meta outage of 2021, WhatsApp recovered more gracefully than other services because its reconnection logic was designed to handle high-concurrency "thundering herd" scenarios where millions of clients attempt to reconnect simultaneously.

Scalability Techniques and Conclusion

The architecture of WhatsApp demonstrates that extreme scale is achieved not by complexity, but by the relentless optimization of simple abstractions.

Technique	Implementation Detail	Result
Vertical Scaling	Optimizing FreeBSD/Erlang to 2M+ connections/host	Fewer servers to manage, reduced coordination latency
Horizontal Sharding	Dividing users across MySQL/Erlang shards based on UserID	Linear scalability for billions of accounts
Stateless Servers	Messages deleted immediately after delivery	Minimal disk I/O, simplified disaster recovery

Technique	Implementation Detail	Result
Asynchronous I/O	Erlang's event-driven runtime and FreeBSD kqueue	High throughput with minimal CPU overhead

In summary, WhatsApp's architecture is built on Erlang/OTP for massive concurrency, FunXMPP for protocol efficiency, and the Signal Protocol for privacy. The deployment on FreeBSD allows for deep networking optimizations, while the transition to Rust for media handling via Kaleidoscope secures the platform against evolving malware threats. The evolution into the MEX layer and IPLS reflects a modernizing infrastructure that balances Meta's integrated ecosystem with WhatsApp's original principles of simplicity and security. For professional peers, WhatsApp serves as a definitive case study in how to build a global-scale utility that is both technologically robust and operationally lean.

Works cited

1. How WhatsApp Scaled to Billions of Users with Just 50 Engineers - Ajit Singh, <https://singhajit.com/whatsapp-scaling-secrets/>
2. Engineering Simplicity: The WhatsApp Story | by Vignaraj Ravi | Medium, <https://medium.com/@vignarajj/engineering-simplicity-the-whatsapp-story-6791fb388edc>
3. WhatsApp Engineering: How End-to-End Encryption Works | by Tanyaradzwa T Mushonga | Jan, 2026 | Medium, <https://medium.com/@tanyaradzwaTMushonga/whatsapp-engineering-how-end-to-end-encryption-works-4d051e68d465>
4. The Architecture Behind WhatsApp: How a 2 Billion User App Achieves Near-Zero Downtime: | by Harshavardhan Mamidipaka | Medium, <https://medium.com/@mamidipaka2003/the-architecture-behind-whatsapp-how-a-2-billion-user-app-achieves-near-zero-downtime-d3ed24a61527>
5. How WhatsApp Handles 40 Billion Messages Daily: A Deep Dive into Minimal Engineering Excellence | by Vikas Kumar | Medium, <https://medium.com/@imvk45/how-whatsapp-handles-40-billion-messages-daily-a-deep-dive-into-minimal-engineering-excellence-2c7be8789d4e>
6. Rust at Scale: An Added Layer of Security for WhatsApp ..., <https://engineering.fb.com/2026/01/27/security/rust-at-scale-security-whatsapp/>
7. Beyond the Scale: The "Why" of WhatsApp's Architectural Simplicity | by Arun Seetharaman, <https://medium.com/@arunseetharaman/beyond-the-scale-the-why-of-whatsapps-architectural-simplicity-43a0e675dbba>
8. WhatsApp Message Delivery Explained | PDF | Communications Protocols | Internet - Scribd, <https://www.scribd.com/document/981986759/How-WhatsApp-Sends-a-Message-in-0>
9. How WhatsApp Works - Architecture Deep Dive on 100 Billion Messages - GetStream.io, <https://getstream.io/blog/whatsapp-works/>
10. Understanding WhatsApp's Architecture & System Design - CometChat, <https://www.cometchat.com/blog/whatsapps-architecture-and-system-design>
11. Performance troubleshooting of FreeBSD networking stack and/or kqueue/kevent, <https://forums.freebsd.org/threads/performance-troubleshooting-of-freebsd-networking-stack-and-or-kqueue-kevent.75153/>
12. FreeBSD TCP Performance System Controls, <https://klarasystems.com/articles/freebsd-tcp-performance-system-controls/>
13. freebsd network tuning - Levent Serinol's Blog, <http://lserinol.blogspot.com/2009/01/freebsd-network-tuning.html>
14. Kernel tuning in FreeBSD, <https://forums.freebsd.org/threads/kernel-tuning-in-freebsd.95264/>
15. FunXMPP Protocol Overview and Utilities | PDF | String (Computer Science) | Xml - Scribd,

<https://www.scribd.com/document/907134476/Comprehensive-FunXMPP-Documentation-Complete-Method-Analysis> 16. Inside Of WhatsApp : Part 1 - Sunny Patel - WordPress.com, <https://mycodeplex.wordpress.com/2016/03/08/inside-of-whatsapp-part-1/> 17. WhatsApp Binary Protocol - WhatsApp Rust - Mintlify, <https://mintlify.com/jlucaso1/whatsapp-rust/advanced/binary-protocol> 18. Welcome to yowsup Discussions! #3027 - GitHub, <https://github.com/tgalal/yowsup/discussions/3027> 19. Double Ratchet Algorithm - Wikipedia, https://en.wikipedia.org/wiki/Double_Ratchet_Algorithm 20. Signal Protocol Projects - AI Tinkerers, <https://sf.aitinkerers.org/technologies/signal-protocol> 21. The Sesame Algorithm: Session Management for ... - Signal, <https://signal.org/docs/specifications/sesame/sesame.pdf> 22. The WhatsApp White Paper | PDF | Key (Cryptography) - Scribd, <https://www.scribd.com/document/958919115/the-WhatsApp-white-paper> 23. WhatsApp Security Whitepaper Preview - JPG | PDF | Encryption | Cryptography - Scribd, <https://www.scribd.com/document/531092710/WhatsApp-Security-Whitepaper-Preview-jpg> 24. How Encrypted Messaging Apps Work: Telegram, Signal, and WhatsApp Compared - Jordan Open Source Association, <https://josa.ngo/blog/288> 25. Signal >> Specifications >> The Double Ratchet Algorithm, <https://signal.org/docs/specifications/doubleratchet/> 26. Diving into Signal's New Post-Quantum Protocol - PQShield, <https://pqshield.com/diving-into-signals-new-pq-protocol/> 27. WhatsApp Multi-Device Support: 2024 Guide - Polser, <https://blog.polser.io/whatsapp-multi-device-support-2024-guide/> 28. WhatsApp System Design: A Complete Architecture Deep Dive | by Yadavsatale - Medium, <https://medium.com/@yadavsatale/whatsapp-system-design-a-complete-architecture-deep-dive-8949f8d4eb2b> 29. How to design WhatsApp like System? : r/softwarearchitecture - Reddit, https://www.reddit.com/r/softwarearchitecture/comments/1ngnuab/how_to_design_whatsapp_like_system/ 30. MyRocks: A space- and write-optimized MySQL database - Engineering at Meta - Facebook, <https://engineering.fb.com/2016/08/31/core-infra/myrocks-a-space-and-write-optimized-mysql-database/> 31. How Does WhatsApp Work? Inside Its Architecture & Design - Ethora, <https://ethora.com/blog/how-does-whatsapp-work-inside-its-architecture-design/> 32. The "MEX" Layer: Inside WhatsApp's EU DMA Compliance ... - Medium, <https://medium.com/@blackloverttech/the-mex-layer-inside-whatsapps-eu-dma-compliance-architecture-67107fb60448> 33. WhatsApp's URL Architecture: The Distributed GraphQL Mesh | by blackloverttech - Medium, <https://medium.com/@blackloverttech/whatsapps-url-architecture-the-distributed-graphql-mesh-cdb2f6f996ad> 34. Messaging Interoperability: WhatsApp enables third-party chats for users in Europe, <https://about.fb.com/news/2025/11/messaging-interoperability-whatsapp-enables-third-party-chats-for-users-in-europe/> 35. IPLS: Privacy-preserving storage for your WhatsApp contacts ..., <https://engineering.fb.com/2024/10/22/security/ipls-privacy-preserving-storage-for-your-whatsapp-contacts/> 36. WhatsApp Unveils Encrypted Contact Manager With Username System | daily.dev, <https://app.daily.dev/posts/whatsapp-unveils-encrypted-contact-manager-with-username-system-dbvcuc37p> 37. WhatsApp Debuts New Features For Contact Management With Enhanced Privacy Protections - Information Security Buzz, <https://informationsecuritybuzz.com/whatsapp-new-features-contact-manageme/> 38. WhatsApp Deploys Rust-Based Media Parser to Block Malware on 3 Billion Devices - InfoQ, <https://www.infoq.com/news/2026/02/whatsapp-rust-media-malware/> 39. 74 - When Meta and Anthropic Choose Rust,

<https://rust-trends.com/newsletter/when-meta-and-anthropic-choose-rust/> 40. WhatsApp Replaces C++ Code To RUST As Shield Billion Users - Hackers Online Club, <https://hackersonlineclub.com/whatsapp-replaces-c-code-to-rust-as-shield/> 41. How WhatsApp Delivers Millions of Messages Around the World in Seconds, <https://digitalterminal.in/trending/how-whatsapp-delivers-millions-of-messages-around-the-world-in-seconds> 42. Whatsapp's storage mechanism, <https://althuraya-yemen.com/uploads/topics/17558042187577.pdf> 43. The System Design of WhatsApp for Android: Behind the Scenes of a Global Messaging Giant | by Yodgorbek Komilov | Medium, <https://medium.com/@YodgorbekKomilo/the-system-design-of-whatsapp-for-android-behind-the-scenes-of-a-global-messaging-giant-c80175b18016> 44. Whatsapp rewrote its media handler to rust (160k c++ to 90k rust) : r/programming - Reddit, https://www.reddit.com/r/programming/comments/1qpdiio/whatsapp_rewrote_its_media_handler_to_rust_160k_c/ 45. AndyTargino/wwebjs-electron: whatsapp-web.js API to electron applications - GitHub, <https://github.com/andytargino/wwebjs-electron> 46. @enderdba/wwebjs-electron - npm, <https://www.npmjs.com/package/%40enderdba%2Fwwebjs-electron> 47. Update: WhatsApp Rust client now has a desktop UI and calls (WIP) - Reddit, https://www.reddit.com/r/rust/comments/1q7hbrw/update_whatsapp_rust_client_now_has_a_desktop_ui/